

Vector Space Models & Word Embeddings

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology



LMH

Lady Margaret Hall

July 2, 2025

① Not fixed length inputs / outputs

① text classification

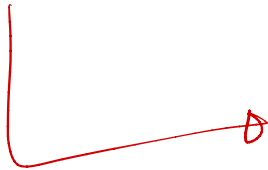
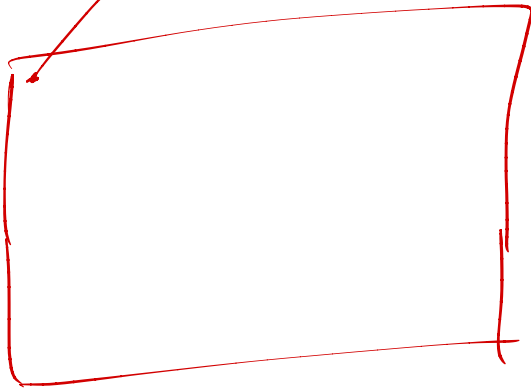
② seq 2 seq

③ language modeling

② Data Representation

CV

{0 ... 255}



NLP

Today is a beautiful day

ASCII

UTF32

ML

Supervised ML

x	y
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
x_N	y_N

Unsupervised ML

x
x_1
$\vdots$
x_N

$x \rightarrow$ data

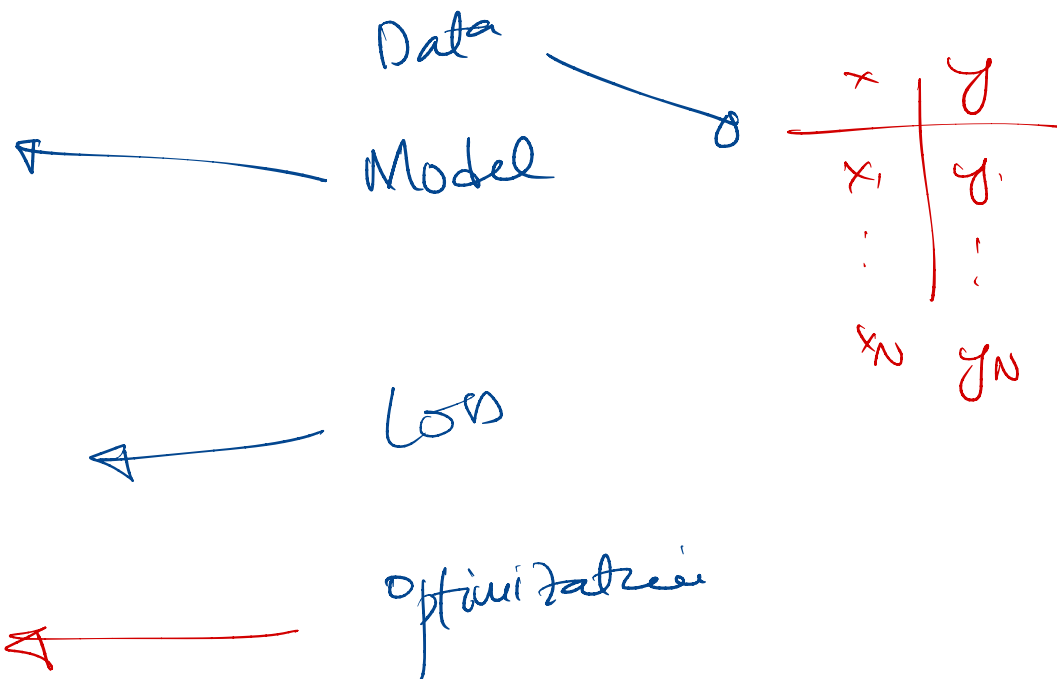
$y \rightarrow$ labels

X_{ij} $\rightarrow$ power
 (k) $\rightarrow$ feature
 X_i $\rightarrow$ data index
 X_{T_i} $\rightarrow$ # of values in a sample
 M $\rightarrow$ total number of feature
 X_N $\rightarrow$ total number of sample

$$\hat{y}_i = \underline{f_{\theta}}(x_i)$$

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N l_i(\hat{y}_i, y_i)$$

$$\arg \min_{\theta} L(\theta)$$



$$\hat{y}_i = f_{\theta}(x_i)$$

$$L(\theta) = \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

$$\nabla_{\theta} L(\theta) = \nabla_{\theta} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

$$= \sum_{i=1}^N \nabla_{\theta} \ell(\hat{y}_i, y_i)$$

↓

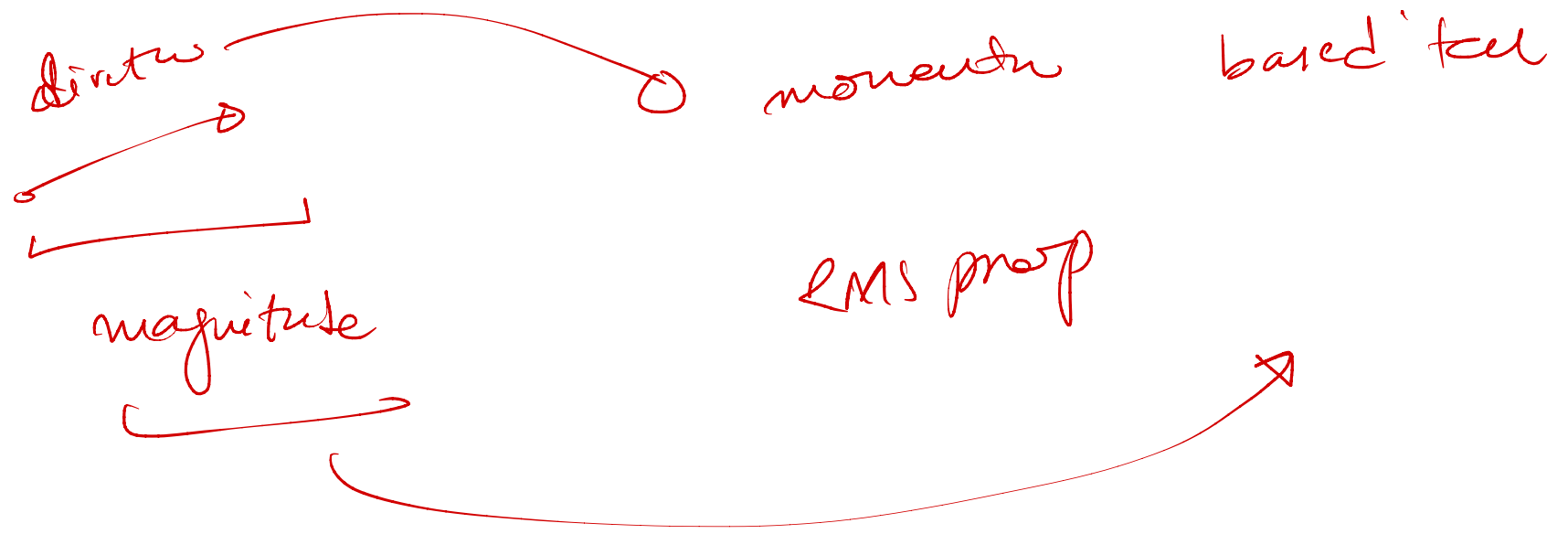
$$10^6 \times 10^7 = 10^{13}$$

stochastic gradient
Descent

x	y
□	1
□	1
□	0
□	0
⋮	
□	1

$$N \rightarrow 10^6$$

$$|\theta| \rightarrow 10^7$$



Example

$$\Theta = \{\omega_1, \omega_2, b\}$$

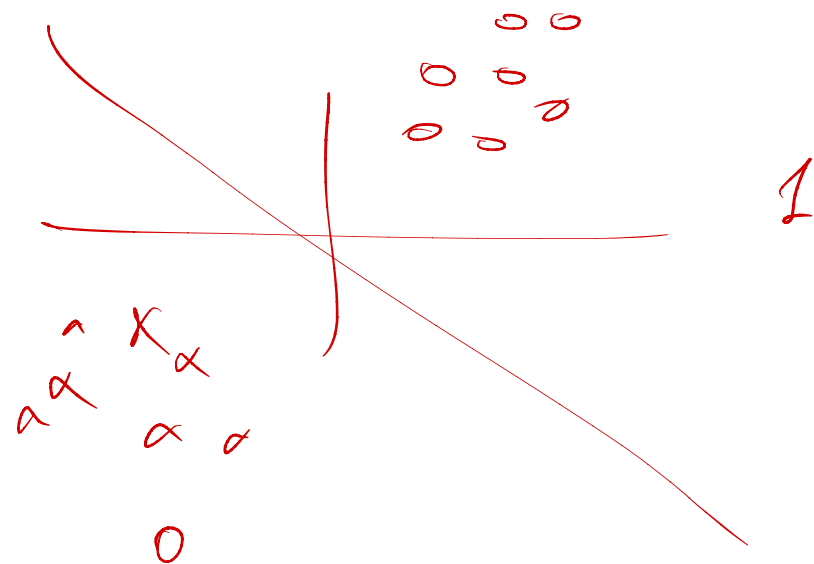
$$\hat{y}_i = f_{\Theta}(x) = b(\omega_1 \underline{x}_i^1 + \omega_2 \underline{x}_i^2 + b)$$

x^1	x^2	y
x_1^1	x_1^2	y_1
x_2^1	x_2^2	y_2
$\vdots$	$\vdots$	$\vdots$
x_n^1	x_n^2	y_n

$$\text{BLE loss} \rightarrow \underline{-y \log \hat{y} - (1-y) \log(1-\hat{y})}$$

$$\mathcal{L} = \sum_{i=1}^N \left(y_i \log \hat{y}_i - (1-y_i) \log(1-\hat{y}_i) \right)$$

$$\nabla_{\Theta} \mathcal{L} = \begin{bmatrix} \underline{\partial \mathcal{L} / \partial \omega_1} \\ \underline{\partial \mathcal{L} / \partial \omega_2} \\ \underline{\partial \mathcal{L} / \partial b} \end{bmatrix} \triangleleft$$



$$\frac{\partial}{\partial w_1} L = \frac{\partial}{\partial w_1} \left(\sum_{i=1}^n (y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)) \right)$$

$$= \sum_{i=1}^n \frac{\partial}{\partial w_1} \left(y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i) \right)$$

$$= \sum_{i=1}^n \frac{y_i}{\hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial w_1}$$

$$\boxed{\frac{\partial \log \hat{y}_i}{\partial w_1} = \frac{\partial \log \hat{y}_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial w_1}}$$

chain rule

$$\frac{\partial L}{\partial \omega_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \omega_1} = \frac{\partial L}{\partial \omega_1}$$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})}$$

$$= \frac{-y(1-\hat{y})}{\hat{y}(1-\hat{y})} + \frac{\hat{y}(1-y)}{\hat{y}(1-\hat{y})}$$

$$= \frac{-y + \cancel{\hat{y}y} + \hat{y} - \cancel{\hat{y}y}}{\hat{y}(1-\hat{y})} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

$$Z = \omega_1 x^1 + \omega_2 x^2 + b$$

$$\hat{y} = \sigma(Z)$$

$$L = -y \log(\hat{y}) - (1-y) \log(1-\hat{y})$$

$$\frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

$$\frac{d}{dz} \left(\hat{y} = 6(z) = \frac{1}{1 + \exp(-z)} = (1 + \exp(-z))^{-1} \right)$$

$$\frac{d}{dz} (1 + \exp(-z))^{-1} = +1 \left[(1 + \exp(-z))^{-2} \right] \exp(-z)$$

$$\frac{\exp(-z) + 1 - 1}{(1 + \exp(-z))^{+2}} = \frac{\cancel{1 + \exp(-z)}}{(1 + \exp(-z))^{\cancel{1} + 2}} - \frac{1^2}{(1 + \exp(-z))^2}$$

$$= \hat{y} - \hat{y}^2 = \hat{y}(1 - \hat{y})$$

Chain rule Example
 $\frac{d}{d\hat{y}} \log(1 - \hat{y})$

$$u = \underline{(1 - \hat{y})}$$

$$\frac{d}{du} \log(u) \cdot \frac{du}{d\hat{y}}$$

$$= \frac{1}{u} (-1) = \frac{1}{(1 - \hat{y})} (-1)$$

$$z = w_1 x^1 + w_2 x^2 + b$$

$$\frac{\partial z}{\partial w_1} = x^1$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \frac{\hat{y}(1-\hat{y})}{\hat{y}(1-\hat{y})} \cdot x^1$$

$$= (\hat{y} - y) \cdot x^1$$

$$\frac{\partial L}{\partial w_2} =$$

$$= (\hat{y} - y) x^2$$

Metrics

$$M(a, b) \longrightarrow \mathbb{R}^+$$

$$M(a, b) \geq 0$$

✓

$$M(a, b) = M(b, a)$$

✗

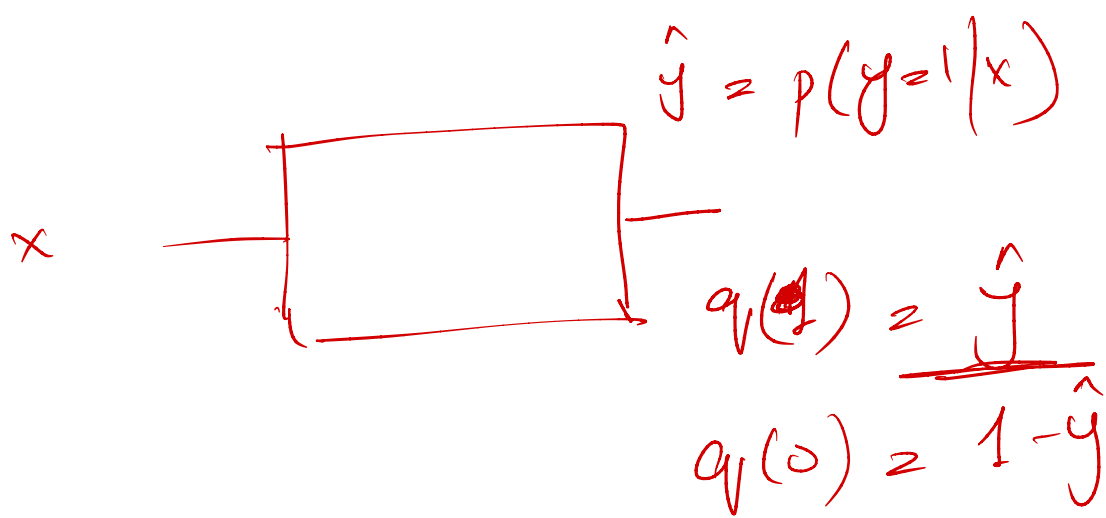
$$M(a, c) \leq M(a, b) + M(b, c)$$

✓

KL divergence

$$K_L(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

$$= \underbrace{\sum_x p(x) \log \frac{p(x)}{q(x)}}_{\text{KL divergence}}$$



$p(1) = 1$

x	y	$p(1)$	$p(0)$
x_1	1	1	0
x_2	1	1	0
$\vdots$	0	0	1
$\vdots$	0	0	1
$\vdots$	1	1	0
x_N	1	1	0

$$KL(p \parallel q) = p(0) \log \frac{p(0)}{q(0)} + p(1) \log \frac{p(1)}{q(1)}$$

$$= - \log(q(0))$$

$$= - \log q(1)$$

BLE loss

$$= - (1-y) \log (1-\hat{y})$$

$$= y \cdot \log(\hat{y})$$

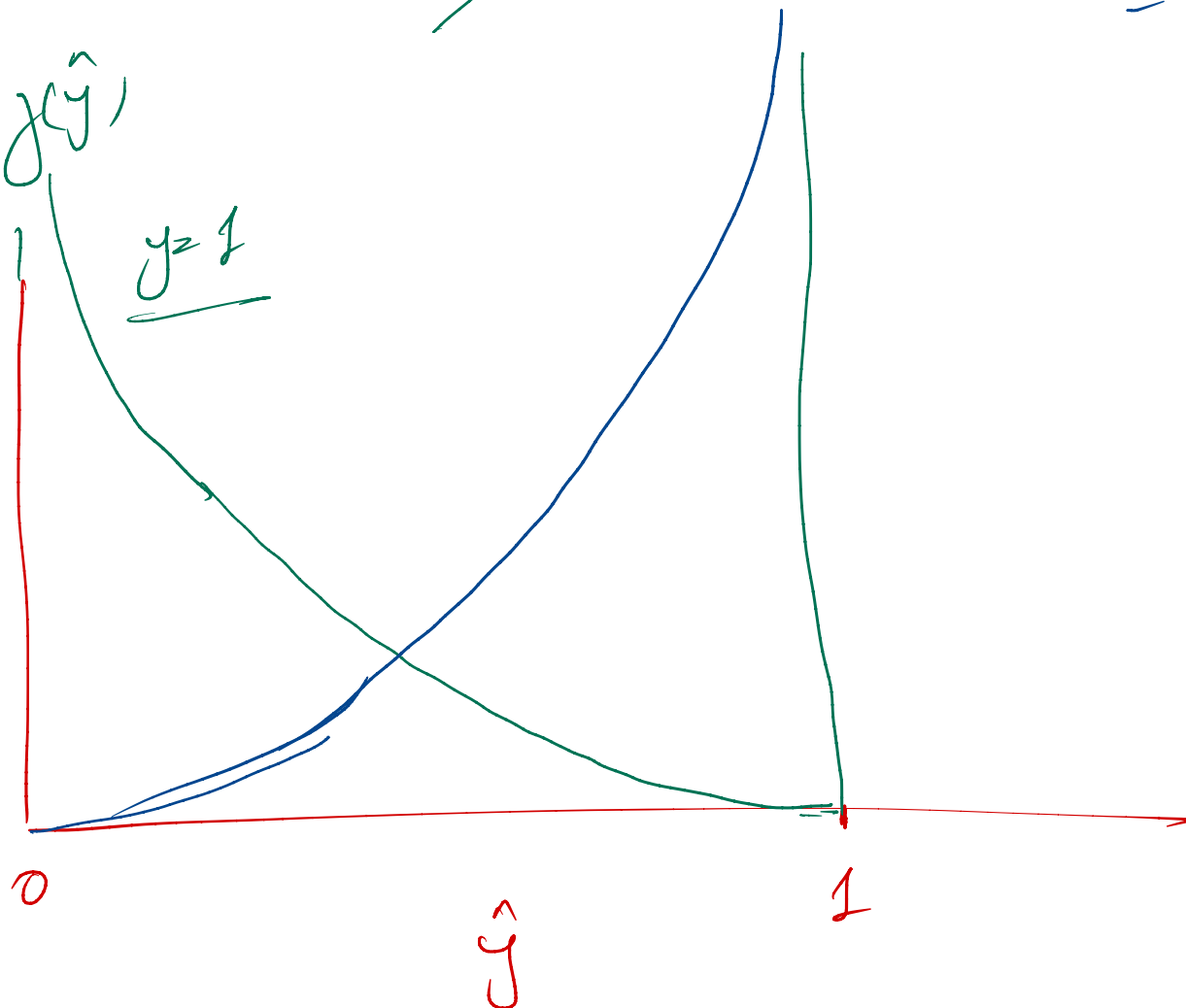
BCE loss

$$\mathcal{L} = -y \log \hat{y}$$

$$-(1-y) \log(1-\hat{y})$$

$$= \log(\hat{y})$$

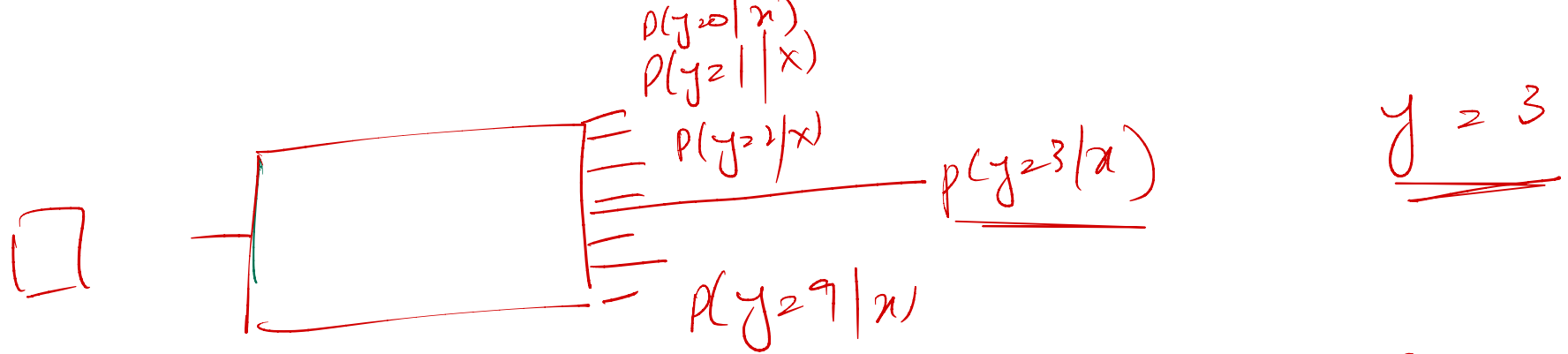
$y=1$



$y \in \{0,1\}$

$$-\log(1-\hat{y})$$

$$y=0$$



$$c \in (j, y) = ?$$

$$- \log(p(y_{23}|x))$$

$$= \sum_{i=1}^n \sum_{j=1}^c \mathbb{1}_{y_i = j} \log(\hat{y}_i^j)$$

Word embeddings

Laplacian $\uparrow$ missing

n-grams

$$p(w^t | w^{t-1})$$

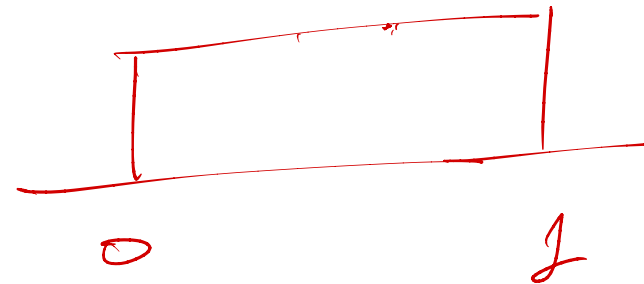
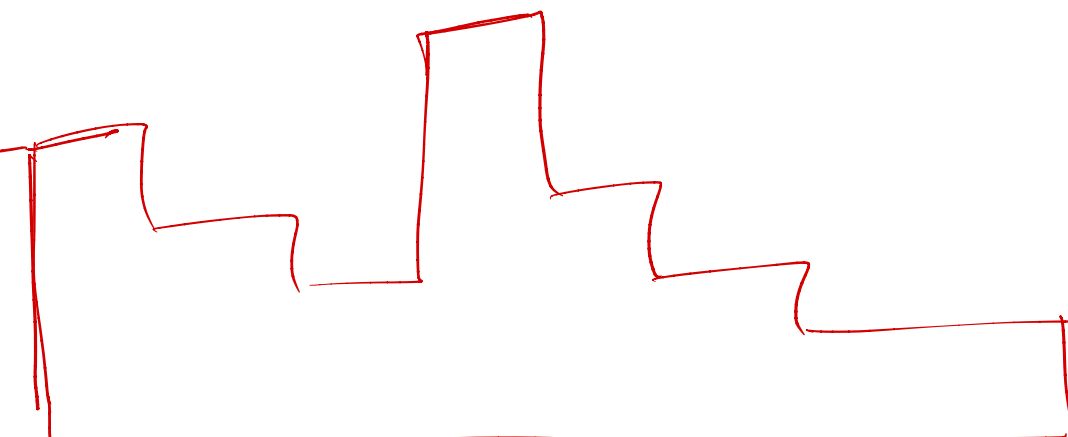
$$= \frac{p(w^t, w^{t-1})}{p(w^{t-1})}$$

$$= \frac{\#(w^t, w^{t-1}) + 1}{\# w^{t-1} + |V|}$$

unigrams

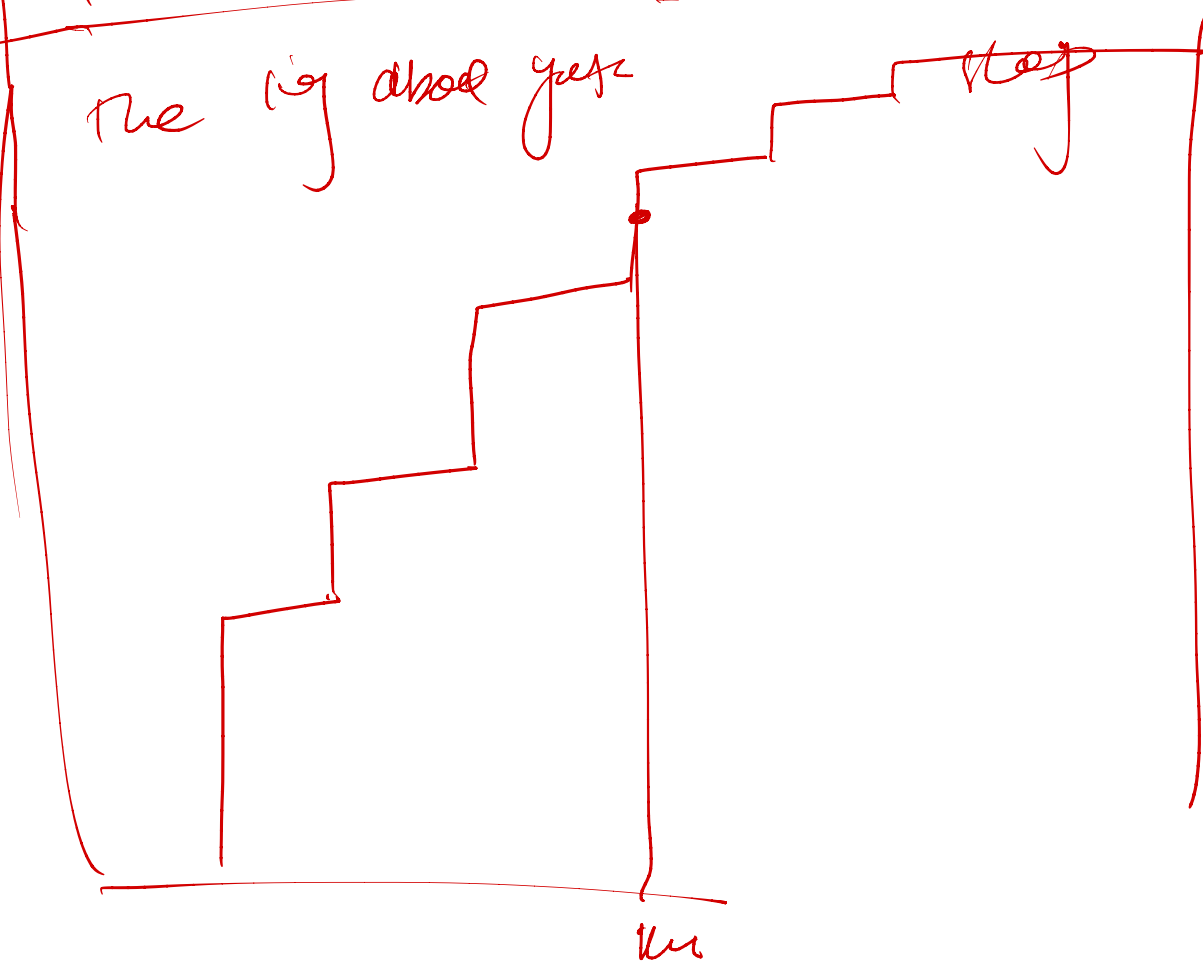
$$p(w^t) = \frac{\# \text{ count of } w^t}{\# \text{ of tokens}}$$

$p(\omega)$
0.2
0.15



The big about just

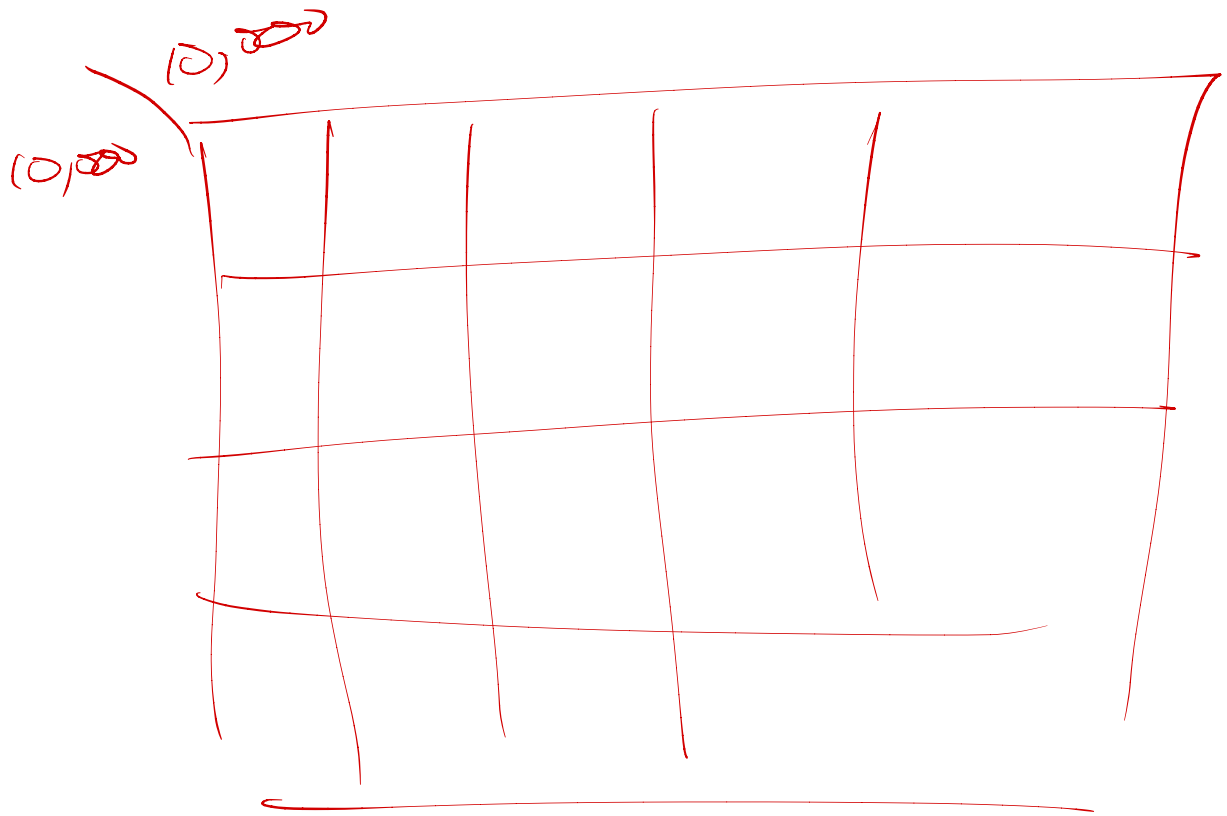
stop



biagram

$|V| = 10,000$

$$p(\omega^1, \omega^2, \dots, \omega^T) = p(\omega^1) \cdot p(\omega^2 | \omega^1) \cdot \underline{p(\omega^3 | \omega^2)} \cdot \underline{p(\omega^4 | \omega^3)} \cdot \underline{p(\omega^T | \omega^{T-1})}$$

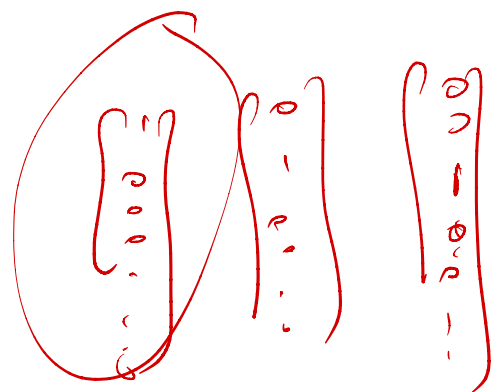
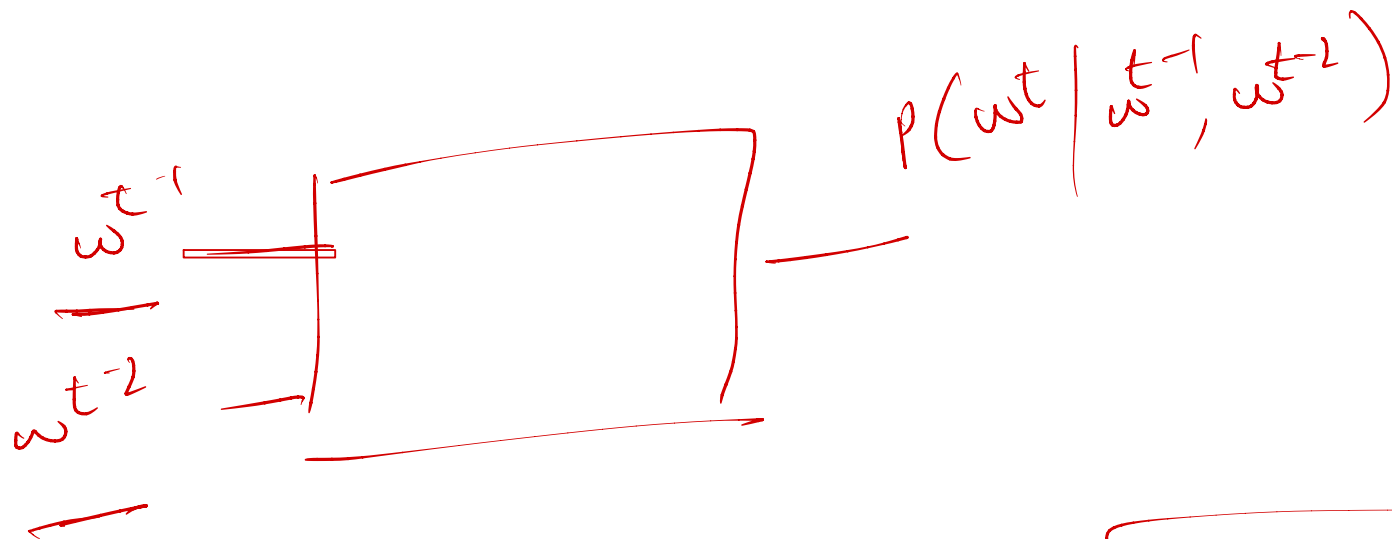


$$P(\omega^t | \omega^{t-1}, \omega^{t-2}) = \frac{\# \text{ of } (\omega^t, \omega^{t-1}, \omega^{t-2}) + 2}{\# \text{ of } (\omega^{t-1}, \omega^{t-2}) + |V|}$$

$$P(\omega^t | \omega^{t-1}, \omega^{t-2})$$

↑

$$\text{hist} \\ P(y|x)$$



1 2 3
 $\{a', a'$ appw



800
 today



800
 beautiful

10,000
 very

Today is a beautiful
 day

w^{t-2}	w^{t-1}	w^t
Today	is	a
is	a	beautiful
a	beautiful	day

apple

orange

table

table

chair

banana

man

woman

cat

ink

pen

carpus

$\bar{e}^T_{apple}$

$\bar{e}_{orange}$

>

$\bar{e}^T_{apple}$

$\bar{e}_{table}$

apple

apple

apple

eat

apple

apple

apple

juice

pie

jam

apple

nachos

feel

stock

orange

orange

orange

orange

orange

juice

peel

cake

map

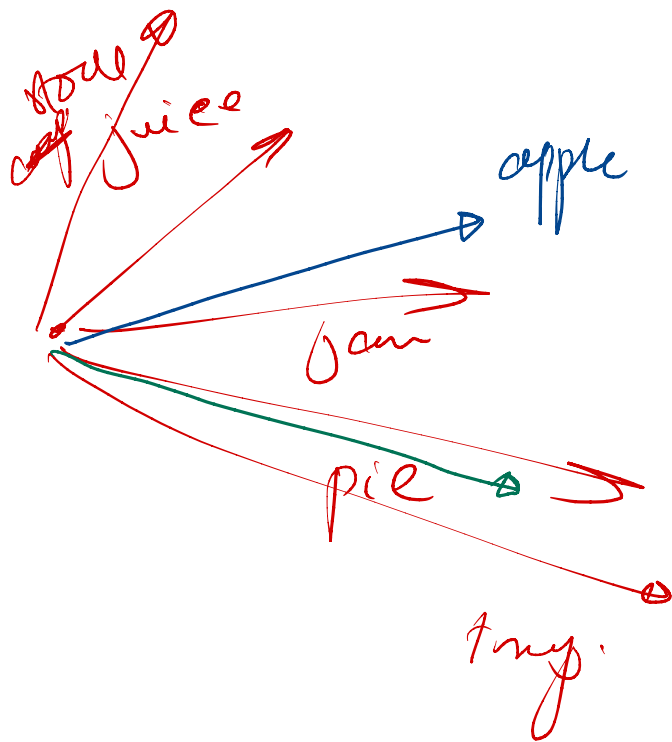
can

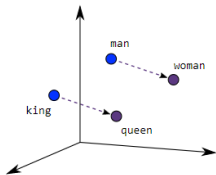
$$\exp(u^T x)$$

$$\sum_u u^T v_u$$

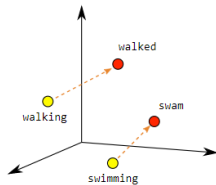
$$2 \rho(w_{\text{out}} / w_{\text{comp}})$$

I eat an apple
 today — u

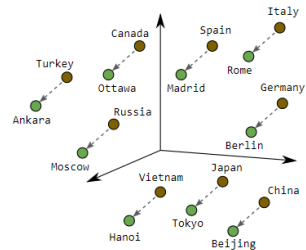




Male-Female



Verb Tense



Country-Capital

1. Motivation
2. Learning Outcomes
3. Introduction
4. Word-by-Word vs Word-by-Document Designs
5. Similarity Measures
 1. Euclidean Distance
 2. Cosine Similarity
 3. Manipulating word vectors
6. Word Embedding: One-Hot vs. Dense Vectors
 1. One-Hot Encoding
 2. Dense Embeddings
7. Embedding Method: Word2Vec

1. Word2Vec – Continuous Bag-of-Words (CBOW)
2. Word2Vec – Skip-Gram
-
-
-
-
-
8. GloVe – Global Vectors
-
9. fastText – Subword Embeddings
-
10. Limitations of Vector Space Models
-
11. Summary
-
12. References

- ▶ Text is symbolic; machines require numerical representations to process it.
- ▶ Classical NLP struggled with sparse, high-dimensional vectors.
- ▶ Vector Space Models (VSMs) and Word Embeddings represent words in a continuous vector space.
- ▶ These models capture semantic meaning and relationships geometrically.
- ▶ Basis for modern NLP methods including Transformers.

- ▶ Understand and implement basic Vector Space Models.
- ▶ Differentiate between word-by-word and word-by-document designs.
- ▶ Use similarity measures to compute semantic relatedness.
- ▶ Explain the rationale behind One-Hot vs. Dense word vectors.
- ▶ Understand and implement basic Word2Vec models (CBOW and Skip-gram).
- ▶ Appreciate the improvements of GloVe and fastText over earlier models.

Vector Space Model: **Introduction**

Why learn vector space models?

Where are you heading?

Where are you from?



Different meaning

What is your age?

How old are you?



Same Meaning

- ▶ Words and sentences can have different meanings depending on context.
- ▶ Vector space models help capture semantic similarity and differences.
- ▶ Useful for tasks like paraphrase detection, question answering, and information retrieval.

“You shall know a word by the company it keeps”
Firth, 1957



(Firth, J. R. 1957:11)

- ▶ VSM represents words or documents as vectors in an n -dimensional space.
- ▶ Based on the **distributional hypothesis**: Words that occur in similar contexts have similar meanings.
- ▶ Forms the foundation for information retrieval, document classification, and word embeddings.

- ▶ A **term-document matrix**: Rows represent words (terms), columns represent documents (or vice versa).
- ▶ Each cell contains a value such as:
 - Term Frequency (TF)
 - TF-IDF (Term Frequency-Inverse Document Frequency)
 - Co-occurrence count
- ▶ The matrix is typically high-dimensional and sparse:
 - $\text{Size} = |\text{Vocabulary}| \times |\text{Documents}|$

- ▶ You eat *cereal* from a *bowl* → Capturing semantic similarity (paraphrase understanding)
- ▶ You *buy* something and someone else *sells* it → Capturing relational meaning (analogies)



Information Extraction



Machine Translation



Chatbots

Word-by-Word vs Word-by-Document Designs

- ▶ Co-occurrence $\rightarrow$ Vector representation
- ▶ Relationships between words/documents

Number of times they *occur together within a certain distance k*

I like simple data
I prefer simple raw data

$k=2$

	simple	raw	like	I
data	2	1	1	0

n

Number of times a word *occurs within a certain category*

	Corpus		
	Entertainment	Economy	Machine Learning
data	500	6620	9320
film	7000	4000	1000



► Word-by-Word:

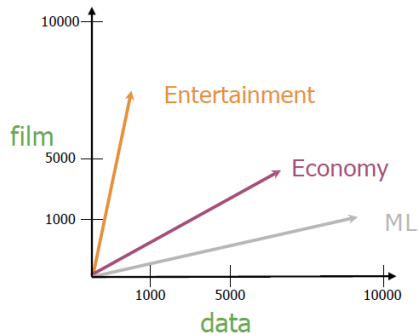
- Matrix built from co-occurrence counts between words.
- Captures semantic similarity directly.
- Better for word similarity tasks.

► Word-by-Document:

- Matrix built from word frequencies in documents.
- Useful for document classification and search.
- Better for document-level tasks.

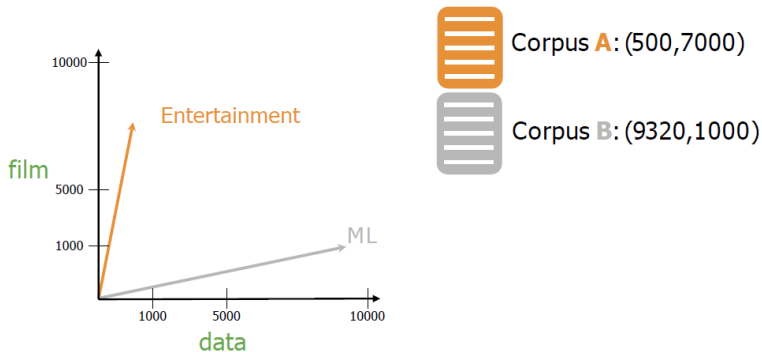
Tradeoffs: Choice depends on the task: word similarity vs. document analysis.

Similarity Measures



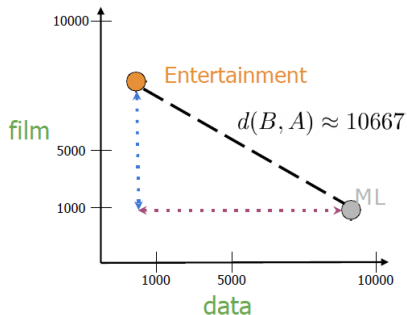
	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of "similarity:"
Angle
Distance



- ▶ Measures the straight-line distance between two points in space.
- ▶ Formula: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- ▶ Sensitive to the scale (magnitude) of the vectors.

Euclidean Distance (cont.)



Corpus **A**: (500,7000)



Corpus **B**: (9320,1000)

$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$
$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(-8820)^2 + (6000)^2}$$

Euclidean Distance for n-dimensional vectors

	data	$\vec{w}$ boba	$\vec{v}$ ice-cream
AI	6	0	1
drinks	0	4	6
food	0	6	8

$$= \sqrt{(1 - 0)^2 + (6 - 4)^2 + (8 - 6)^2}$$

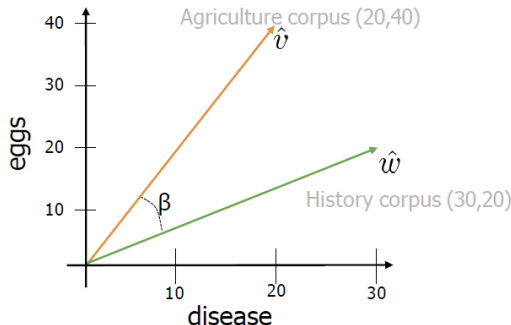
$$= \sqrt{1 + 4 + 4} = \sqrt{9} = 3$$

$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^n (v_i - w_i)^2} \longrightarrow \text{Norm of } (\vec{v} - \vec{w})$$

```
# Create numpy vectors v and w
v = np.array([1, 6, 8])
w = np.array([0, 4, 6])

# Calculate the Euclidean distance d
d = np.linalg.norm(v-w)
# Print the result
print("The Euclidean distance between v and w is: ", d)
```

The Euclidean distance between v and w is: 3



$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

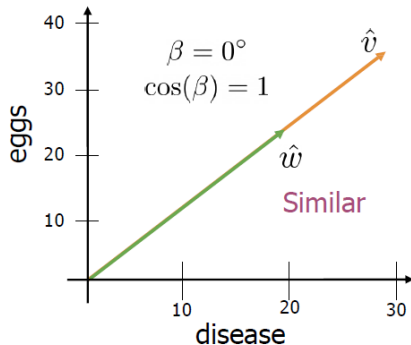
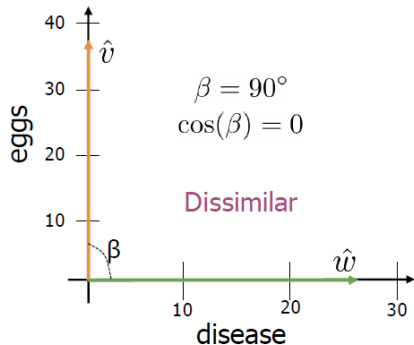
$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}} = 0.87$$

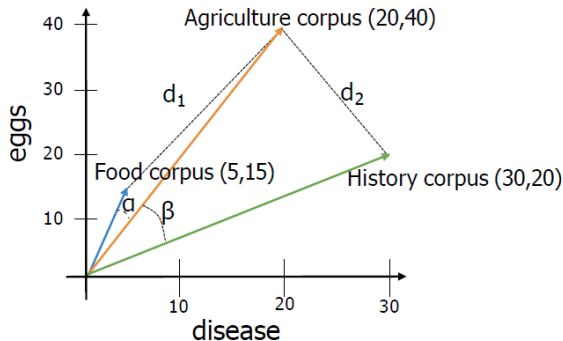
- Measures the cosine of the angle between two vectors.
- Formula: $\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$
- Ranges from -1 (opposite) to 1 (same direction).

- ▶ Less sensitive to magnitude, focuses on orientation.

Cosine Similarity (cont.)



- Cosine similarity when corpora are different sizes



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle between the vectors

Manipulating word vectors



USA



Washington
DC

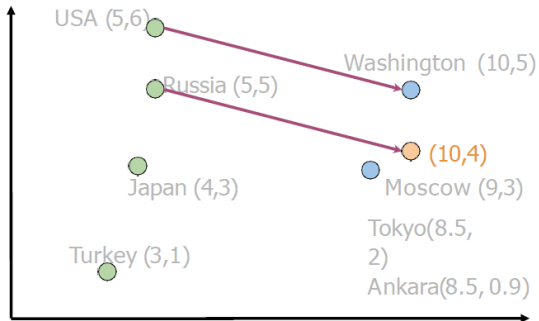


Russia



?

Manipulating word vectors (cont.)



$$\text{Washington} - \text{USA} = \begin{bmatrix} 5 & -1 \end{bmatrix}$$

$$\text{Russia} + \begin{bmatrix} 5 & -1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \end{bmatrix}$$



Moscow

[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

	$d > 2$		
oil	0.20	...	0.10
gas	2.10	...	3.40
city	9.30	...	52.1
town	6.20	...	34.3

How can you visualize if your representation captures these relationships?



oil & gas

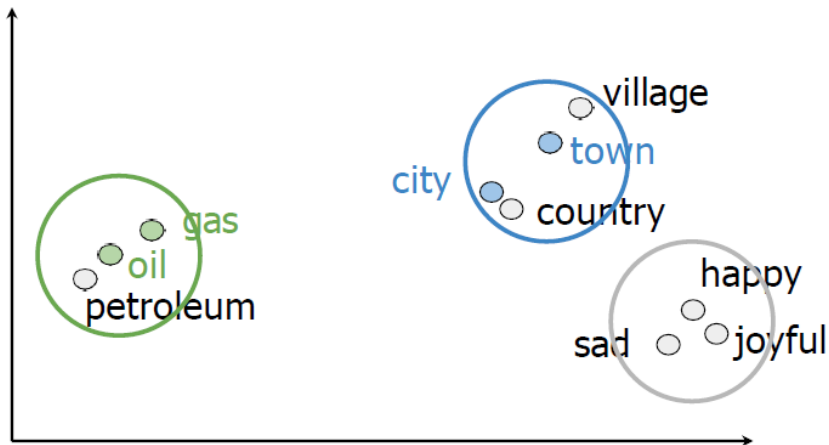


town & city

Visualization of word vectors (cont.)

$d > 2$							
oil	0.20	...	0.10	PCA	oil	2.30	21.2
gas	2.10	...	3.40		gas	1.56	19.3
city	9.30	...	52.1		city	13.4	34.1
town	6.20	...	34.3		town	15.6	29.8

Visualization of word vectors (cont.)



Word Embedding: **One-Hot vs. Dense Vectors**

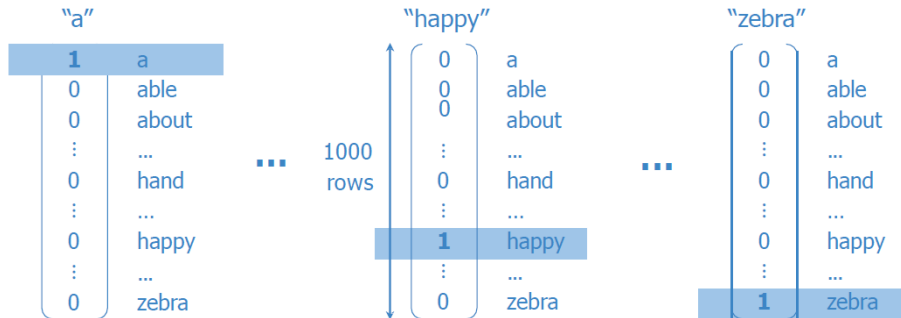
Integer Representation of Words

Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

- + Simple
- Ordering: little semantic sense

hand < happy < zebra
615 621 1000
?! ?!

One-hot vectors



One-hot vectors (cont.)

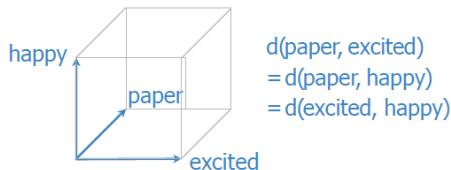
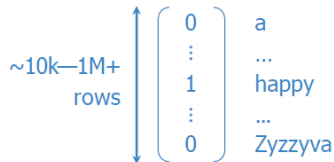
Word	Number			
a	1			
able	2			
about	3			
...	...			
hand	615			
...	...			
happy	621	←	621	→
...	...			
zebra	1000			

"happy"

1 0 a
2 0 able
3 0 about
...
615 0 hand
...
621 1 happy
...
1000 0 zebra

One-hot vectors (cont.)

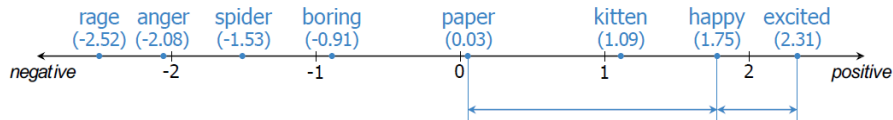
- + Simple
- + No implied ordering
- Huge vectors
- No embedded meaning



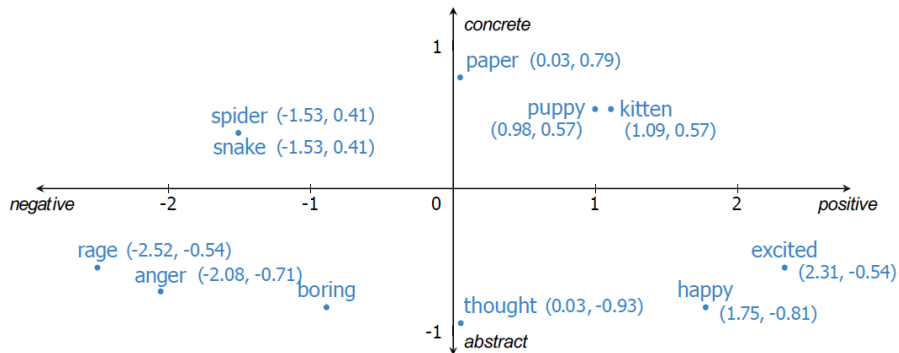
- ▶ Each word is represented as a unique vector.
- ▶ Vector has 1 at one position, 0 elsewhere.
- ▶ **Problems:**
 - High-dimensional and sparse.
 - No semantic meaning or similarity captured.

- ▶ Vectors are learned from data.
- ▶ Low-dimensional (e.g., 100–300).
- ▶ Capture semantic relationships.
- ▶ Example: "king" – "man" + "woman" $\approx$ "queen"

Meaning as vectors



Meaning as vectors (cont.)



+ Low dimension

+ Embed meaning

- e.g. semantic distance

forest $\approx$ tree forest $\not\approx$ ticket

- e.g. analogies

Paris:France :: Rome:?

"happy"

$\sim 100 \text{---} \sim 1000$
rows

$$\begin{pmatrix} 0.123 \\ \vdots \\ -4.059 \\ \vdots \\ 1.891 \end{pmatrix}$$

integers

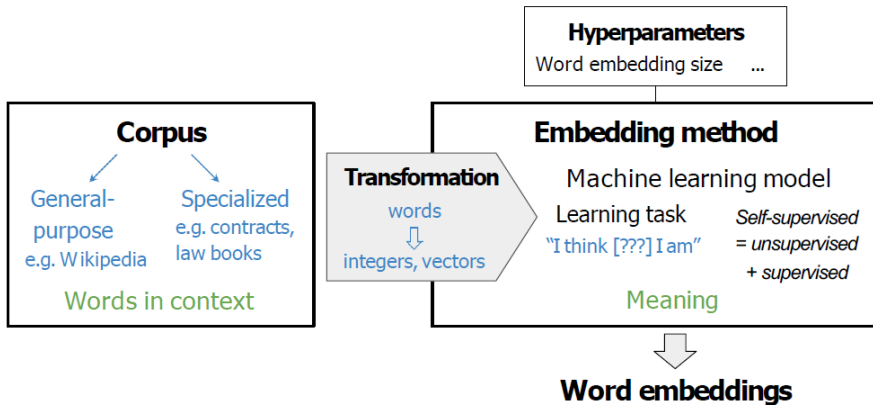
word vectors

one-hot vectors

word embedding vectors

"word vectors"

word embeddings



Embedding Method: **Word2Vec**

- ▶ Dense vector representations of words learned from context.
- ▶ Encode syntactic and semantic meaning.
- ▶ Vectors reflect relationships like synonymy, analogy, etc.

- ▶ **word2vec** (Google, 2013)
 - Continuous bag-of-words (CBOW)
 - Continuous skip-gram / Skip-gram with negative sampling (SGNS)
- ▶ **Global Vectors (GloVe)** (Stanford, 2014)
- ▶ **fastText** (Facebook, 2016)
 - Supports out-of-vocabulary (OOV) words

- ▶ Deep learning-based, contextual embeddings:
 - **BERT** (Google, 2018)
 - **ELMo** (Allen Institute for AI, 2018)
 - **GPT-2** (OpenAI, 2018)
- ▶ Tunable pre-trained models available

- ▶ Word2Vec learns word embeddings from large text corpora.
- ▶ Two main architectures:
 - Continuous Bag-of-Words (CBOW)
 - Skip-gram
- ▶ Both models use neural networks to predict words based on context.

- ▶ **Goal:** Predict target word from context.
- ▶ **Architecture:**
 - **Input:** Surrounding words (context)
 - **Output:** Target word
- ▶ Fast and accurate for frequent words.
- ▶ **Example:**
 - Context: “the ___ sat on the mat” → Predict “cat”

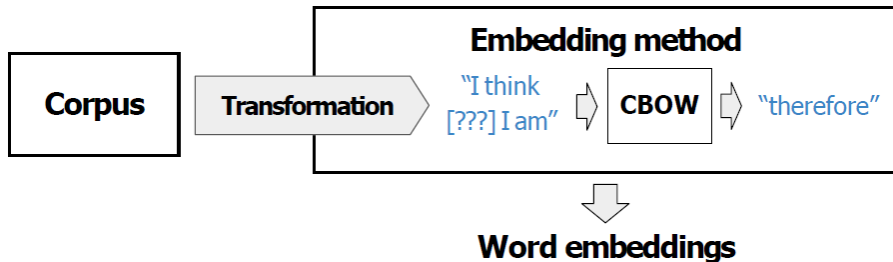


Figure 2: CBOW Architecture



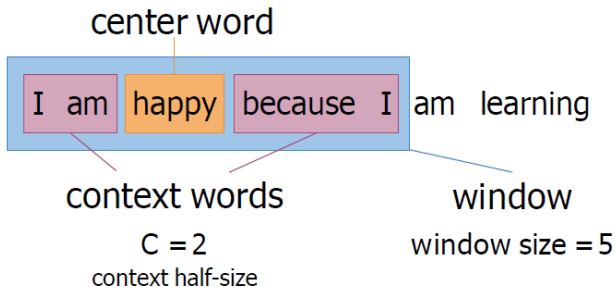
The little _____ ? _____ is barking

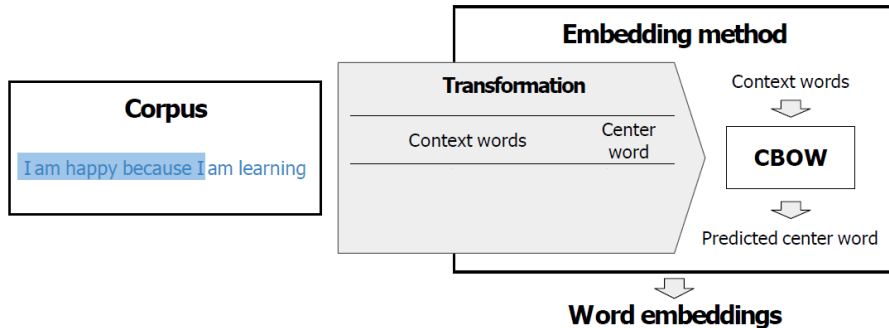


dog
puppy
hound
terrier

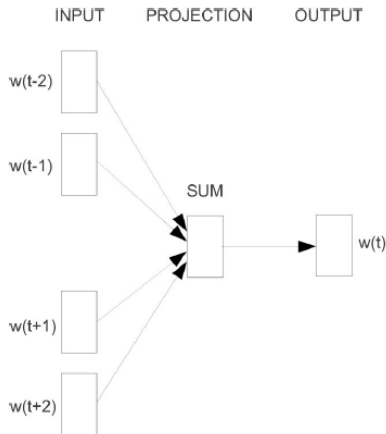
...

Creating a training example



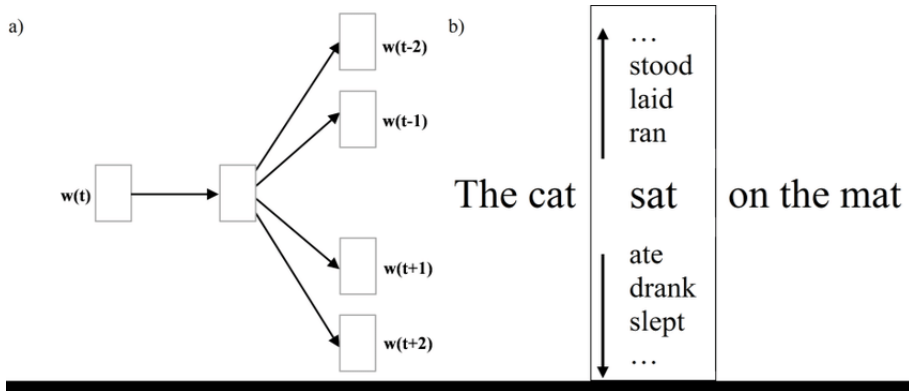


- ▶ **Goal:** Predict context words from a target word.
- ▶ **Architecture:**
 - **Input:** Target word
 - **Output:** Surrounding words (context)
- ▶ Better for rare words.
- ▶ **Example:**
 - Input: “cat” → Output: “the”, “sat”, “on”, “the”, “mat”



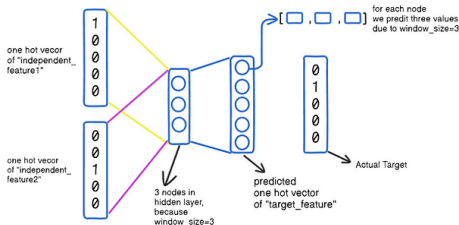
Source: Mikolov, T., Chen, K.,
Corrado, G.S., & Dean, J. (2013).
[Efficient Estimation of Word
Representations in Vector Space](#)

Figure 3: Skip-Gram Architecture

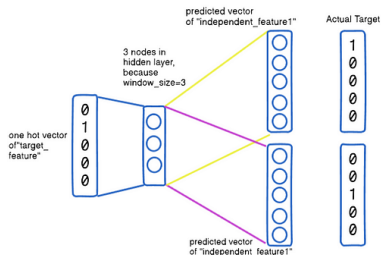


Word2vec

CBOW



Skip-gram



- ▶ Does not consider sub-word information.
- ▶ Same vector for all senses of a word (polysemy issue).
- ▶ Ignores word order within the context window.

GloVe – Global Vectors

Proposed by: Pennington et al., 2014

- ▶ Combines global matrix factorization with local context windows.
- ▶ Captures co-occurrence statistics of words across entire corpus.
- ▶ Embeddings reflect ratios of co-occurrence probabilities.

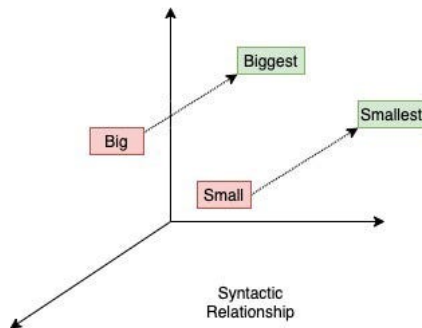
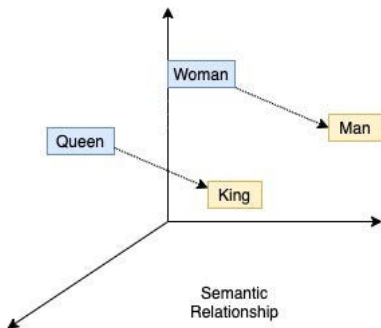
Loss function:

$$J = \sum_{i,j=1}^V f(P_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

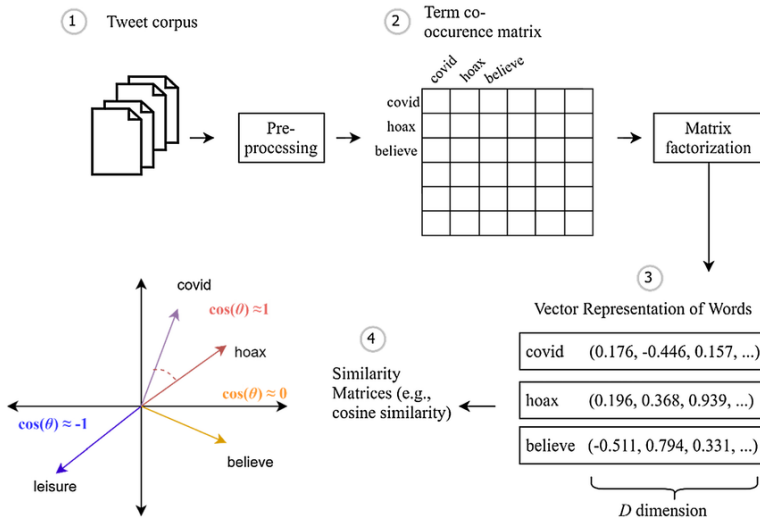
where:

- ▶ X_{ij} : number of times word j occurs in the context of word i
- ▶ P_{ij} : probability of word j in the context of i
- ▶ $w_i, \tilde{w}_j$: word and context word vectors
- ▶ $b_i, \tilde{b}_j$: bias terms
- ▶ f : weighting function

GloVe – Global Vectors (cont.)



GloVe – Global Vectors (cont.)





fastText – Subword Embeddings

- ▶ Developed by Facebook AI (2016)
- ▶ Builds on Word2Vec by incorporating character n-grams
- ▶ Useful for morphologically rich languages and rare words
- ▶ Better handling of OOV (out-of-vocabulary) words

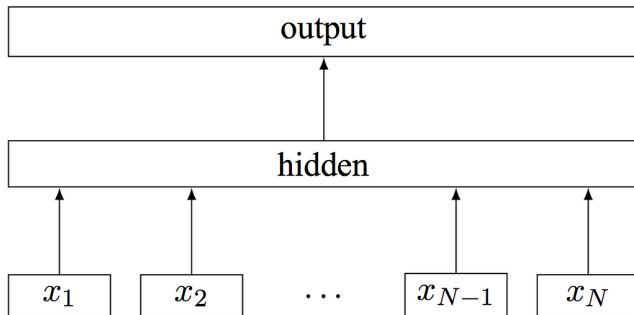
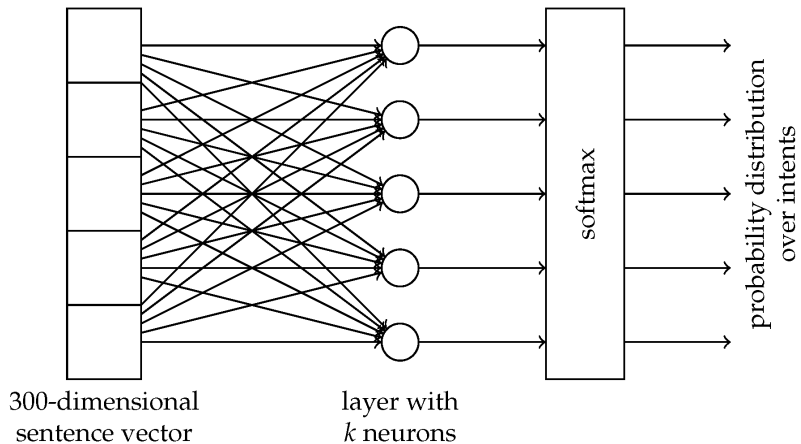


Figure 1: Model architecture of `fastText` for a sentence with N ngram features $x_1, \dots, x_N$. The features are embedded and averaged to form the hidden variable.



FastText-Based Intent Detection for Inflected Languages

- ▶ Word: playing
- ▶ Character n-grams (n=3): ["pla", "lay", "ayi", "yin", "ing"]
- ▶ Word vector = sum of n-gram vectors

Comparison: Word2Vec vs. GloVe vs. fastText

Feature	Word2Vec	GloVe	fastText
Local Context	✓	—	✓
Global Info	—	✓	✓ (partial)
Subword Info	—	—	✓
Handles OOV	—	—	✓

► High Dimensionality:

- Vectors can become very high-dimensional, leading to computational inefficiency.
- Curse of dimensionality: distance metrics become less meaningful.

► Sparsity:

- One-hot vectors are sparse, leading to inefficiencies in storage and computation.
- Dense vectors mitigate this but still require large datasets for effective training.

► Lack of Context:

- Traditional VSMs do not capture word context effectively.
- Same word can have different meanings in different contexts (polysemy).

► Semantic Limitations:

- Cannot capture complex relationships like negation or antonymy.
- Similar words may not always be semantically related (e.g., "bank" vs. "river bank").

► Scalability:

- As vocabulary size increases, the term-document matrix becomes larger and more sparse.
- Requires significant computational resources for training and inference.



Summary

- ▶ **Contextual embeddings** (ELMo, BERT, GPT): Word vectors depend on sentence context.
- ▶ **Multilingual embeddings** and cross-lingual models.
- ▶ **Graph-based embeddings** (e.g., knowledge graph completion).
- ▶ **Hybrid embeddings**: Combining structured and unstructured data.

- ▶ Vector Space Models (VSMs) are foundational for modern NLP.
- ▶ Word embeddings capture semantic relationships in a continuous space.
- ▶ Word2Vec, GloVe, and fastText are key methods for generating word vectors.
- ▶ Dense embeddings outperform one-hot vectors in capturing meaning and relationships.
- ▶ Future work focuses on contextual, multilingual, and hybrid embeddings.



References

- [1] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).
Efficient Estimation of Word Representations in Vector Space.
arXiv:1301.3781.
- [2] Pennington, J., Socher, R., & Manning, C. D. (2014).
GloVe: Global Vectors for Word Representation.
EMNLP.
- [3] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017).
Enriching Word Vectors with Subword Information.
TACL.
- [4] Jurafsky, D., & Martin, J. H. (2023).
Speech and Language Processing (3rd Ed Draft).

- [5] Stanford NLP Slides.
<https://web.stanford.edu/class/cs224n>

- [6] Facebook AI Research (FAIR) – fastText.
<https://fasttext.cc>

- [7] Goldberg, Y. (2016).
A Primer on Neural Network Models for NLP.
JMLR.

- [8] Chrupała, G. (2019).
Symbolic and Subsymbolic Representations in NLP – Deep Learning Lectures.

Credits

Dr. Prashant Aparajeya

Computer Vision Scientist — Director(AISimply Ltd)

p.aparajeya@aisimply.uk

This project benefited from external collaboration, and we acknowledge their contribution with gratitude.